BỘ MÔN EMBEDDED LINUX SYSTEMS

LOS201 — IC2009

BÁO CÁO THIẾT KẾ VÀ PHÂN TÍCH HỆ THỐNG SENSOR MONITORING

ASSIGNMENT 02 • TECHNICAL REPORT

THÔNG TIN	NỘI DUNG
Sinh viên	Trần Tuấn Minh
Mã số sinh viên	SE203134
Môn học	Embedded Linux Systems
Môi trường	Ubuntu x86-64 · QEMU ARMv7 · Linux 5.15
Phiên bản báo cáo	Bản FINAL kỹ thuật (cập nhật kết quả QEMU 116 mẫu), 27/10/2026

Báo cáo dựa trên source và log do sinh viên cung cấp; các giới hạn dữ liệu được công khai tại Mục 17.

TỔNG QUAN

Tóm tắt điều hành

Bài Assignment 02 triển khai hệ thống giám sát cảm biến nhiệt độ/độ ẩm trên Linux nhúng. Kernel module cung cấp character device, ioctl và procfs; ứng dụng userspace gồm bốn worker thread dùng circular buffer, POSIX message queue và cơ chế graceful shutdown.

Báo cáo tập trung vào bằng chứng thực nghiệm: Task 1 đã ghi 113 mẫu trong một phiên trước, sau đó từng xuất hiện hồi quy 78 mẫu/59,36 giây ở bản hardening. Sau bản sửa retry khi driver trả EAGAIN, kiểm thử cuối trên QEMU ARM đạt 116 mẫu/59,54 giây tại chu kỳ 500 ms, exit=0 và overflow=0; read_count cũng tăng 116. Task 2 kiểm chứng Valgrind và bộ nhớ; Task 3 ghi nhận GDB, strace và perf; Task 4 so sánh SCHED_OTHER/FIFO tại chu kỳ benchmark 100 ms (không đồng nhất với phép thử chức năng 500 ms).

CHỈ SỐ NỔI BẬT	GIÁ TRỊ
Valgrind x86 mock	17 allocations / 17 frees; 0 bytes at exit; 0 errors
Driver ARM, 60 giây (Task 2)	read_count tăng 113; ioctl 500 ms OK
VmSize / VmRSS (5/30/55s)	35.596 kB / 4 kB, không đổi trong log
GDB	5 threads; backtrace tất cả; snapshot buffer dưới mutex
Live perf	1 sample, không đủ xếp hạng hotspot
Jitter P99 OTHER / FIFO	9.939 / 9.736 µs (giảm 2,04%)
Lỗi và log rotation	normal exit=0; injected logger failure exit=1; rotation PASS
Retest 500 ms — bản source đã sửa	116/116 mẫu; 59,54 s; read_count +116; exit=0; overflow=0

> KẾT QUẢ XÁC NHẬN CUỐI CÙNG: Sau cập nhật cơ chế retry EAGAIN tại Sensor Thread, phiên QEMU chạy -i 500 -d 60 thu được 116/116 mẫu, size=0, overflow=0, exit=0, uptime=59,54 giây; read_count tăng đúng 116. Kết quả đạt dải yêu cầu 110–130 mẫu. Kết quả cũ 78 mẫu được giữ để giải thích quá trình phát hiện và xử lý hồi quy.

Tài liệu gồm kiến trúc hệ thống, phân tích đồng bộ, thiết kế kernel/userspace, bảng đo, biểu đồ, thảo luận giới hạn, kế hoạch demo và reflection.

CHỈ DẪN

Mục lục và sơ đồ nguồn chứng cứ

01. 1. Mục tiêu, phạm vi và phương pháp
02. 2. Kiến trúc tổng thể và trạng thái
03. 3. Thiết kế kernel module sms_sensor
04. 4. Thiết kế bốn worker threads
05. 5. Shared buffer, IPC và đồng bộ hóa
06. 6. Task 1: chức năng, shutdown, độ tin cậy
07. 7. Task 2: quản lý bộ nhớ và tích hợp driver
08. 8. Task 3: GDB, strace và perf
09. 9. Task 4: scheduling và jitter
10. 10. Phân tích giới hạn và cải tiến
11. 11. Demo video và hướng dẫn tái lập
12. 12. Reflection, kết luận và phụ lục

Nguyên tắc diễn giải

- Kết quả QEMU ARM và kết quả Host x86 giả lập sensor được phân biệt rõ; Host mock không thay thế kiểm chứng kernel driver.
- Các số đo CPU của perf không được suy diễn thành tỷ lệ hotspot nếu số samples quá ít.
- VmRSS=4 kB và VmSize=35.596 kB được ghi đúng theo /proc/PID/status, cần nhận định trong phạm vi QEMU.
- Phiên bản source trong archive có 6 file .c userspace và driver kernel; phần kỹ thuật gắn với mã nguồn được kiểm tra, không dựa vào phỏng đoán.

NỀN TẢNG

1. Mục tiêu, tiêu chí và phương pháp

1.1. Mục tiêu kỹ thuật

Ứng dụng sms_app thu mẫu từ /dev/sms_sensor theo chu kỳ, tính trung bình trượt, phát cảnh báo vượt ngưỡng, ghi nhật ký và kết thúc an toàn khi SIGINT hoặc hết thời gian. Driver phải cho phép điều chỉnh khoảng lấy mẫu qua SMS_SET_INTERVAL và cung cấp /proc/sms_stats.

1.2. Môi trường và công cụ

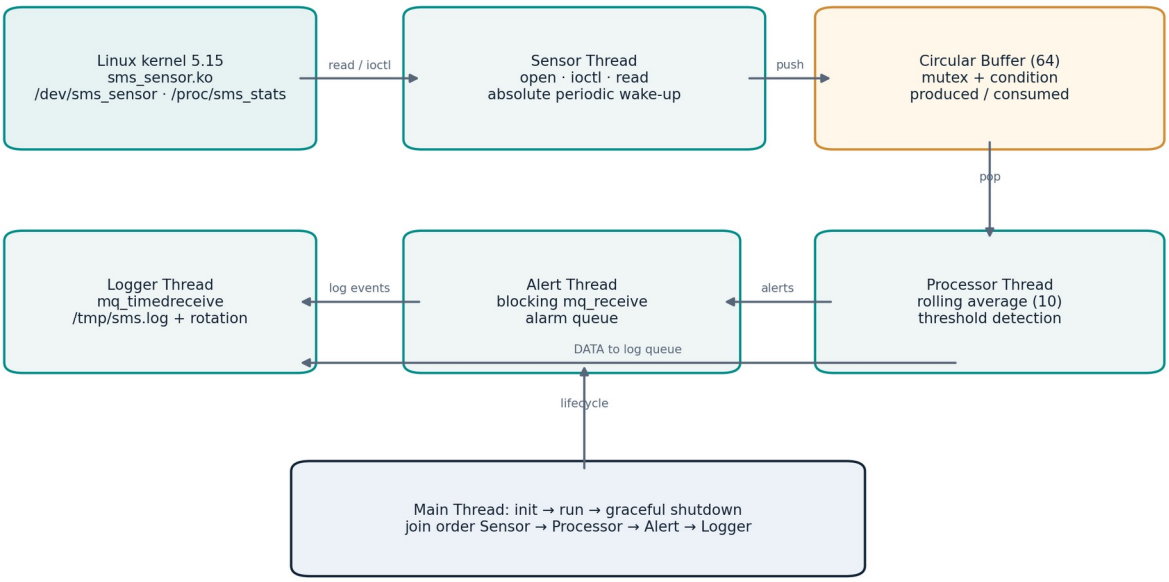
THÀNH PHẦN	CẤU HÌNH/DIỄN GIẢI
Host	Ubuntu x86-64; GCC, GDB, strace, Valgrind, perf
Target	QEMU vexpress-a9 / Cortex-A9; kernel Linux 5.15, ARMv7 EABI5
Cross-compiler	arm-linux-gnueabihf-gcc; ARM app -static; host app ELF x86-64
Synchronization	pthread mutex/condition; C11 atomic; POSIX mqueue
Debug/profiling	GDB multithread, strace -f -c, perf record/stat
Real-time	SCHED_OTHER và SCHED_FIFO priority 50; MONOTONIC absolute sleep

1.3. Phương pháp thực nghiệm

1. Kiểm thử chức năng (driver + app) trong QEMU bằng kernel module thực.
2. Đo bộ nhớ và thực hiện Valgrind trên Host bằng mock sensor tách biệt.
3. Dùng GDB/strace/perf để quan sát thread, syscall, sampling overhead.
4. So sánh jitter theo hai chính sách scheduler; yêu cầu n>=1000 mẫu mỗi chính sách.
5. Kiểm tra lỗi: log path invalid, queue send timeout, rotate file >1 MiB.

THIẾT KẾ HỆ THỐNG

2. Kiến trúc tổng thể



Hình 1. Dòng dữ liệu và quan hệ các worker threads (thiết kế được kiểm tra trong source).

2.1. Đường dữ liệu

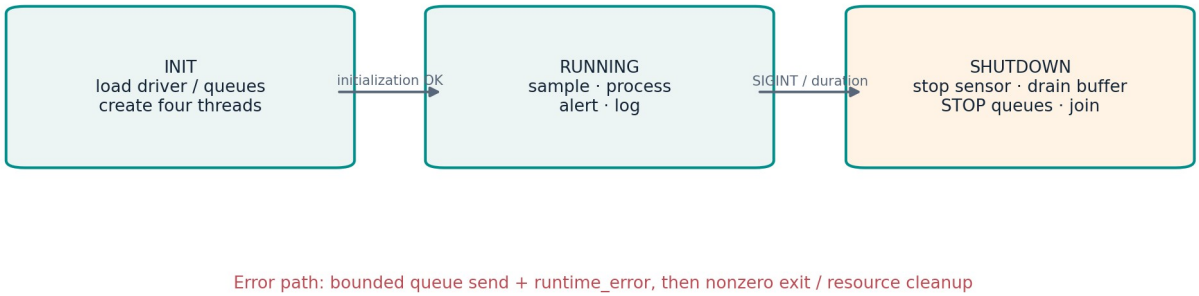
Dữ liệu bắt đầu từ sms_sensor.ko. Sensor Thread đọc qua thiết bị ký tự, chuyển chuỗi CSV thành SensorData rồi push vào CircularBuffer. Processor Thread pop mẫu, tính rolling average và phát chuỗi DATA/ALERT. Alert Thread nhận message từ alert_queue, hiển thị stderr và chuyển cảnh báo tới log_queue. Logger Thread ghi nội dung từ log_queue vào /tmp/sms.log.

2.2. Lý do lựa chọn

- Circular buffer kích thước cố định 64 giúp kiểm soát memory footprint và tránh malloc/free liên tục khi đang chạy.
- POSIX message queues phân tách thời gian xử lý cảnh báo và ghi file, làm giảm phụ thuộc trực tiếp giữa worker.
- Logger Thread riêng giúp thực thi rotate và flush mà không chặn bước đọc sensor ở luồng khác, dù queue đầy vẫn có thể tạo backpressure.

THIẾT KẾ HỆ THỐNG

2.3. State machine và lifecycle



Hình 2. Trạng thái ứng dụng: INIT → RUNNING → SHUTDOWN; đường lỗi thoát với nonzero.

2.4. Thứ tự khởi tạo

Ở main(), tiến trình mở device, gọi ioctl đặt interval, khởi tạo circular buffer rồi tạo hai message queues. Bốn worker được tạo theo thứ tự Logger → Alert → Processor → Sensor (ngược chiều phụ thuộc) nhằm giảm nguy cơ phát dữ liệu khi downstream chưa hoạt động.

2.5. Thứ tự shutdown

6. Main gắn stop_requested; Sensor Thread ngừng phát mẫu; pthread_join(sensor).
7. cb_close() đặt cờ closed và broadcast condition variable; Processor drain buffer rồi kết thúc.
8. Main gửi STOP vào alert_queue khi không còn producer alert; join Alert.
9. Main gửi STOP vào log_queue khi không còn producer log; join Logger và flush file.
10. Chỉ sau join mới in R3 counters: produced, consumed, size, overflow.

Thứ tự này bảo toàn tính toàn vẹn thông điệp trong điều kiện bình thường; tình huống lỗi consumer được xử lý qua timeout và fallback cancel.

THIẾT KẾ DRIVER

3. Kernel module sms_sensor

3.1. Giao diện và vòng đời

Driver gọi alloc_chrdev_region → cdev_add → class_create → device_create để đăng ký /dev/sms_sensor, đồng thời tạo proc entry sms_stats. Trình giải phóng thực hiện ngược lại: proc_remove, device_destroy, class_destroy, cdev_del và unregister_chrdev_region.

3.2. API ioctl

```
#define SMS_IOC_MAGIC 'S'
#define SMS_SET_INTERVAL _IOW(SMS_IOC_MAGIC, 1, unsigned int)
```

sms_ioctl chỉ xử lý SMS_SET_INTERVAL; lệnh không biết trả -ENOTTY. Driver copy_from_user và từ chối 0 hoặc >60000 ms bằng -EINVAL. interval_ms được thay đổi trong vùng bảo vệ của sms_lock.

3.3. Read, EOF, EAGAIN

Mỗi lần open mới có offset ban đầu bằng 0. read với *ppos != 0 trả 0 (EOF). Nếu hai lượt read quá gần nhau (<interval_ms) thì trả -EAGAIN. Khi đủ chu kỳ, driver sinh nhiệt độ 15–45°C và độ ẩm 30–90% dạng fixed point x100, xây chuỗi timestamp,temp,humidity, rồi copy_to_user.

3.4. Procfs và kế thừa R1/R2

TRƯỜNG /proc/sms_stats	Ý NGHĨA
read_count	Số read thành công cập nhật thống kê
total_bytes_read	Tổng số byte thực sự trả cho userspace (R1)
last_temperature / humidity	Giá trị mẫu gần nhất
min_temperature / max_temperature	Cực trị từ khi module được nạp (R2), N/A trước mẫu đầu
interval_ms / uptime	Chu kỳ cấu hình và thời gian module tồn tại

sms_lock bảo vệ các bộ đếm cùng cực trị. Kernel không dùng số thực; định dạng XX.yy chỉ được xây khi in/ghi userspace.

THIẾT KẾ USERSPACE

4. Thiết kế ứng dụng đa luồng

4.1. Sensor Thread

Chờ sensor_run_gate từ main để scheduler policy được áp dụng trước khi bắt đầu; dùng clock_gettime(CLOCK_MONOTONIC) và clock_nanosleep(TIMER_ABSTIME) tạo mốc đánh thức tuyệt đối. Đọc thiết bị bằng open/read/close, parse chuỗi bằng sscanf, push vào shared buffer và đo jitter wake-up vào mảng 4.096 phần tử.

4.2. Processor Thread

Lấy mẫu qua cb_pop có blocking condition; lưu 10 nhiệt độ và độ ẩm gần nhất trong mảng vòng để tính avg_t, avg_h. Khi vượt 40°C, 80% hoặc chênh nhiệt độ liên tiếp >5°C sẽ phát TEMP_HIGH, HUMID_HIGH hoặc TEMP_SPIKE vào alarm_queue. Các ngưỡng 40/80 có thể đổi bằng -t và -h.

4.3. Alert Thread

Sử dụng blocking mq_receive() để nhận alarm; phân loại cảnh báo thành 3 counter. Mỗi cảnh báo được in ra stderr và chuyển tới Logger Thread. STOP marker kết thúc sau khi Processor đã dừng, tránh mất log ở đường bình thường.

4.4. Logger Thread

Ghi timestamp đầy đủ dạng [YYYY-mm-dd HH:MM:SS ±HHMM] vào /tmp/sms.log. mq_timedreceive() chờ tối đa một giây mỗi lượt để vẫn flush theo chu kỳ khoảng 5 giây. File được rotate khi ftell()>1 MiB thành sms.log.1, sau đó tạo file mới.

4.5. CLI và trạng thái

```
sms_app [-i ms] [-t °C] [-h %] [-d sec] [-v] [-r fifo|other]
-i 500 -t 40 -h 80 -d 60 -v   # ví dụ Task 1
```

PHÂN TÍCH THREAD SAFETY

5. Shared data và synchronization

5.1. Ma trận dữ liệu dùng chung

SHARED DATA	NGƯỜI TRUY CẬP	BẢO VỆ
CircularBuffer items/head/tail/size	Sensor, Processor, Main getters	pthread_mutex_t
Buffer nonempty / closed	Sensor, Processor, Main	pthread_cond_t + mutex
stop_requested / runtime_error	Main và worker threads	atomic_int
processor_done / alert_done	Main, lifecycle	atomic_int
Alert queues	Processor → Alert; Alert → Logger	POSIX mqueue + STOP marker
Counters alert_*	Alert writes; Main reads after join	Lifecycle / join (single writer)
jitter_count/values	Sensor writes; Main reads after join	Lifecycle / join (single writer)

5.2. Trình tự khóa và tránh deadlock

cb_push/cb_pop chỉ giữ buffer mutex khi sửa dữ liệu vòng; mọi câu lệnh mq_timedsend được gọi từ Processor sau khi cb_pop đã trả về, tức không giữ buffer mutex trong lúc đợi message queue. Điều này ngăn chu trình lock(buffer) → wait(queue) → lock(buffer) trong đường bình thường.

Consumer chạy độc lập và STOP messages chỉ được gửi sau khi thread phát nguồn đã join. Mỗi mq_timedsend có giới hạn 2 giây; nếu log consumer đã hỏng, atomic flag giúp sender trả EPIPE và main đánh dấu runtime_error.

> Giới hạn: dữ liệu chứng minh được các kịch bản đã thử (normal, SIGINT, injected logger error). Không thể coi thử nghiệm này là chứng minh hình thức cho mọi interleaving, mọi lỗi đĩa hay mọi scheduler.

KẾT QUẢ THỰC NGHIỆM

6. Task 1 — Chạy chức năng và độ ổn định

6.1. Kết quả các phiên 500 ms

PHIÊN ĐO	DURATION	PRODUCED / CONSUMED	OVERFLOW	ĐÁNH GIÁ
Baseline trên QEMU	≈60 s	113 / 113	0	Đạt dải 110–130
Blocking queue (SIGINT)	≈13.38 s	26 / 26	0	Exit=0; 5 threads
Bản hardening trước sửa EAGAIN	59.36 s	78 / 78	0	Hồi quy; chưa đạt dải 110–130
Bản cuối sau retry EAGAIN	59,54 s	116 / 116	0	PASS: dải 110–130, exit=0

Lần chạy ban đầu tại interval=500 ms đạt 113/113 mẫu trong khoảng 60 giây; bài SIGINT trên phiên blocking queues đạt 26/26 và overflow=0, với 5 threads đang hoạt động khi quan sát. Bản hardening từng ghi 78 mẫu/59,36 giây (thấp hơn tiêu chí). Sau khi điều chỉnh Sensor Thread thử lại thao tác read khi nhận EAGAIN, phép chạy cuối trên QEMU ARM đạt 116/116 mẫu, size=0, overflow=0; ứng dụng exit=0, uptime=59,54 giây, driver read_count tăng 116, interval_ms=500. Phép thử mới nhất đáp ứng yêu cầu 120±10 mẫu.

6.2. Hồi quy 78 mẫu và xác nhận sau khi sửa

- Phiên QEMU cũ từng thu thấp hơn mục tiêu: 78 mẫu trong 59,36 giây so với dải 110–130 mẫu ở khoảng 60 giây. Đây là kết quả thực, được giữ lại để trình bày quá trình điều tra hồi quy.
- Một nguyên nhân kỹ thuật hợp lý là read trả EAGAIN khi driver và Sensor Thread đánh thức theo hai mốc thời gian độc lập ở chu kỳ 500 ms; bản cũ bỏ qua lần đọc đó. Bản cập nhật retry EAGAIN trong giới hạn phù hợp đã được biên dịch lại và cài vào initramfs QEMU. Phép kiểm chứng mới đạt 116 mẫu; chưa có log đếm chi tiết EAGAIN để chứng minh đây là nguyên nhân duy nhất.
- Bằng chứng kết quả cuối: exit=0; [SUMMARY] produced=116 consumed=116 size=0 overflow=0; [SUMMARY] alerts_total=124 TEMP_HIGH=16 HUMID_HIGH=26 TEMP_SPIKE=82 uptime_sec=59.54; /proc/sms_stats trước/sau cho delta read_count=116, interval_ms=500; min_temperature=15.07°C, max_temperature=44.32°C. Ảnh Terminal gốc đặt trong Phụ lục C.

Tính nhất quán được kiểm tra: 116 = 116 + 0 và 16 + 26 + 82 = 124. Lần chạy này đạt tiêu chí 110–130 mẫu; việc chứng minh ổn định trong nhiều phiên/tải khác nhau vẫn cần thêm thí nghiệm nếu triển khai thực tế.

KẾT QUẢ THỰC NGHIỆM

6.3. Task 1 — Cảnh báo, logging và shutdown

6.3.1. Cảnh báo và tính nhất quán

Trong phiên mới nhất đạt 116 mẫu, hệ thống ghi nhận 124 cảnh báo: TEMP_HIGH=16, HUMID_HIGH=26 và TEMP_SPIKE=82; tổng bằng 124. Ứng dụng in hai dòng [SUMMARY] sau khi join threads: dòng thứ nhất chứa produced/consumed/size/overflow, dòng thứ hai ghi số lượng alert theo loại và uptime=59,54 giây. Phiên trước (113 mẫu) có 129 cảnh báo, được giữ trong log như một phép thử độc lập.

6.3.2. Kiểm thử quay vòng log

Bằng chứng final_fixes_qemu_session.txt: sau khi file vượt 1 MiB, logger tạo sms.log.1 kích thước 1.049.689 bytes và sms.log mới 855 bytes. Test tự động báo ROTATION_PASS. Đây là test có chủ đích đẩy vượt ngưỡng, không phải đo log phát triển tự nhiên trong 60 giây.

6.3.3. Kiểm thử đường lỗi

KỊCH BẢN	EXIT / OUTPUT	DIỄN GIẢI
Host mock bình thường	exit 0	Không regress xử lý chính
Host mock log path hỏng	exit 1	Báo lỗi thay vì treo
QEMU Logger I/O failure	ERROR_PATH_NONZERO_PASS	Đường lỗi có nonzero
SIGINT với blocking queue	exit 0; size=0; overflow=0	Stop messages, join và flush hoạt động

Bản sửa dùng mq_timedsend với absolute deadline 2 giây. Nếu consumer hỏng hoặc queue bị nghẽn vượt deadline, hàm đặt runtime_error/stop_requested và main dùng fallback cancel khi không thể gửi STOP. Đây là hardening, không thay thế giám sát lỗi cấp hệ thống.

MEMORY MANAGEMENT

7. Task 2 — Phân tích Valgrind

7.1. Đối tượng và phương pháp

Do Host x86-64 không có ARM character driver /dev/sms_sensor, Valgrind chạy ứng dụng sms_app logic bốn thread với backend mô phỏng cảm biến. Các mutex, condition, POSIX mqueue, alerts và logger vẫn nằm trong binary được kiểm tra. Việc giả lập sensor cần được nêu rõ trong phương pháp thực nghiệm.

7.2. Kết quả leak-check

CHỈ SỐ	KẾT QUẢ
Công cụ	Valgrind 3.18.1 / Memcheck
Command	task2/sms_app_x86_mock -i 500 -d 0
Heap allocations	17
Heap frees	17
Total allocated	16.139 bytes
In use at exit	0 bytes / 0 blocks
Valgrind errors	0 errors in 0 contexts

> HEAP SUMMARY: All heap blocks were freed — no leaks are possible. ERROR SUMMARY: 0 errors from 0 contexts.

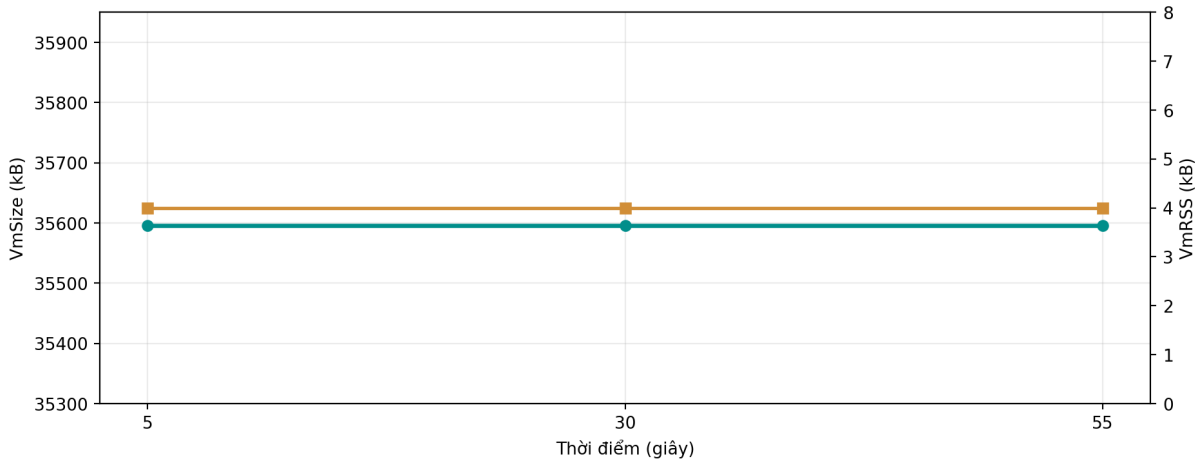
Log cũng liệt kê 4 file descriptors (3 standard) tại exit, trong đó FD 3 là valgrind_sms.txt được thừa kế từ parent log-redirection. Dữ liệu này không tự động chứng minh mọi FD khác đều đóng đúng trong mọi failure path; song lần chạy này không còn heap allocations tồn đọng.

7.3. Giải thích

CircularBuffer.items được cấp phát một lần bằng calloc trong cb_init(), hủy bằng cb_destroy() sau khi các worker join. POSIX queue được mq_close và mq_unlink. Logger fflush/fclose. Sự cân bằng alloc/free trên Valgrind là bằng chứng trực tiếp mạnh về bản test này.

MEMORY MANAGEMENT

7.4. Task 2 — Memory footprint



Hình 3. VmSize và VmRSS tại 5, 30, 55 giây theo QEMU /proc/PID/status.

THỜI ĐIỂM	VmSize	VmRSS	VmData	THREADS
5 s	35.596 kB	4 kB	33.188 kB	5
30 s	35.596 kB	4 kB	33.188 kB	5
55 s	35.596 kB	4 kB	33.188 kB	5

Trong ba ảnh chụp /proc/PID/status này, không xuất hiện xu hướng tăng VmSize hoặc VmRSS. VmSize biểu thị không gian địa chỉ ảo đã ánh xạ, khác với RSS (resident pages). Giá trị VmRSS 4 kB rất thấp với chương trình ARM static và phụ thuộc QEMU/kernel reporting; nên không diễn giải thành mức RAM thực tế của toàn bộ ứng dụng/guest.

7.5. Các nguyên nhân có thể khiến footprint ổn định

- Circular buffer dung lượng 64 × sizeof(SensorData) được cấp phát một lần; không tăng theo số mẫu.
- Rolling-average histories là mảng cố định 10 phần tử trên stack của Processor Thread.
- Message queues có mq_maxmsg=10 và kích thước thông điệp 192 bytes, giới hạn độ dài hàng đợi.

MEMORY MAP

7.6. Task 2 — /proc/PID/maps

File process_maps.txt của QEMU cho thấy nhiều virtual memory areas. Bảng dưới mô tả năm nhóm region theo đúng dữ liệu thu thập:

VÙNG ĐỊA CHỈ	QUYỀN	Ý NGHĨA
00010000–00082000	r-xp	Text: mã lệnh của /usr/bin/sms_app
00092000–00094000	r--p	Dữ liệu chỉ đọc / metadata của executable
00094000–00096000	rw-p	Data và BSS của executable
00096000–000bb000	rw-p	[heap] cấp phát động
74df5000–755f5000	rw-p	Vùng anonymous lớn, phù hợp stack worker
7ee85000–7eea6000	rw-p	[stack] của main thread
7eeec000–7eeed000	r-xp	[vdso] cho một số dịch vụ kernel/user

Các vùng ---p đứng cạnh các vùng anonymous lớn hoạt động như guard pages cho thread stacks; trong /proc/PID/maps không nhất thiết xuất hiện nhãn [stack:<tid>] đối với từng thread. Kiểm chứng stack từng worker cần đọc /proc/<pid>/task/<tid>/maps tương ứng.

> Bản ARM của sms_app được liên kết tĩnh, nên không có các .so thông thường của chương trình dynamic. Yêu cầu “shared libraries” trong bài được giải thích bằng sự khác biệt này; không bịa thêm vùng thư viện không tồn tại.

DRIVER TESTING

8. Task 2 — Tích hợp kernel driver

8.1. Tests ioctl và read

CASE	EXPECTED	ACTUAL
ioctl 500 ms	Success	[OK] ioctl 500ms
ioctl interval = 0	-EINVAL	[OK] ioctl 0 -> EINVAL
Unknown ioctl	-ENOTTY	[OK] unsupported ioctl -> ENOTTY
First read	One CSV sample	[OK] 17 bytes
Second read on same FD	EOF	[OK] EOF
Immediate second open/read	-EAGAIN	[OK] EAGAIN

8.2. Procfs trước và sau bài chạy

STATISTIC	TRƯỚC	SAU
read_count	1	114
total_bytes_read	17	2050
interval_ms	500	500
min_temperature	16.73	15.26
max_temperature	16.73	45.00

Kết quả Task 2 trước đây có delta read_count=114−1=113, thuộc dải 110–130 ở khoảng 60 giây, interval=500 ms. Phép chạy cuối sau khi sửa Sensor Thread cũng ghi nhận delta read_count=116 (0→116) và produced=116, xác nhận tích hợp driver–userspace trên bản cuối đạt mục tiêu chức năng. Hai phiên thuộc các thời điểm/build khác nhau, không phải cùng một lần đo.

8.3. Kiến trúc error handling

Kernel sử dụng -EAGAIN khi chưa đủ interval, -ENOTTY cho ioctl lạ, -EINVAL cho chu kỳ không hợp lệ, -EFAULT nếu copy_to_user/copy_from_user thất bại. Userland bỏ qua EAGAIN/EINTR khi đọc và xử lý lỗi nặng bằng runtime_error và dừng.

DEBUGGING & PROFILING

9. Task 3 — GDB debugging

9.1. Scenario A: observe shared buffer under mutex

GDB chạy phiên bản Host x86 mock để tận dụng symbol -g. Breakpoint đặt trong cb_pop tại circular_buffer.c:62, trong vùng mutex. Lệnh info threads nhận diện 5 threads; thread apply all bt cung cấp backtrace cho main, logger, alert, processor và sensor.

9.2. Snapshot

STATE	VALUE
head	0
tail	1
current_size	1
total_produced	1
total_consumed	0
capacity	64

Bảng chứng tỏ một mẫu mới đã được ghi vào tail trong khi Processor chuẩn bị pop. Quan sát tại breakpoint bảo vệ bởi mutex tránh trạng thái dữ liệu không nhất quán khi đọc đồng thời.

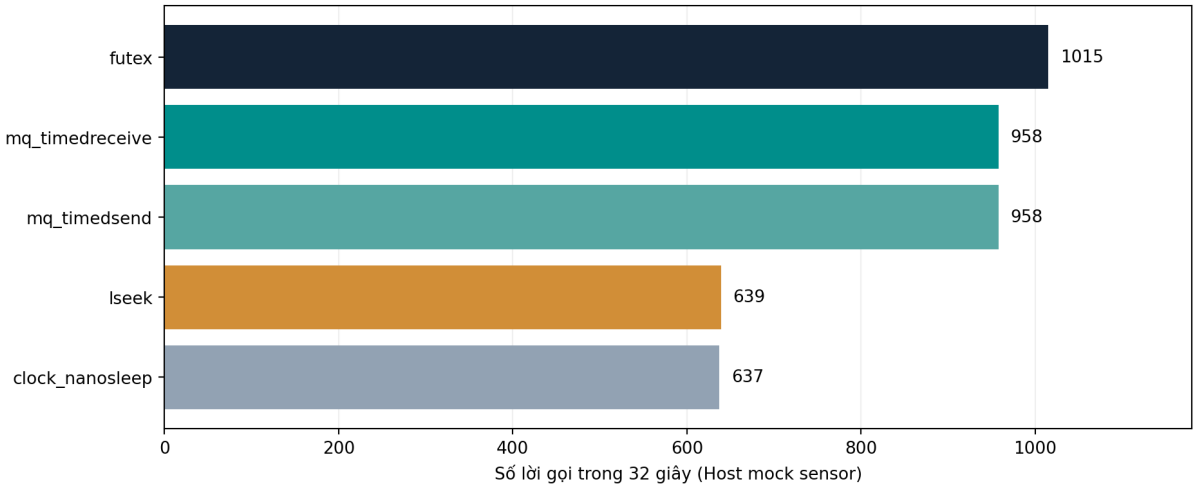
9.3. Thread call stacks

- Main: nanosleep / ms_sleep trong vòng giám sát thời gian.
- Sensor: ms_sleep chuẩn bị lấy mẫu; các trạng thái thời gian phụ thuộc bản Task 3 trước Task 4.
- Processor: cb_pop và processor_thread; điểm break quan trọng nhất.
- Alert: blocking mq_receive trong alarm queue.
- Logger: timed receive, có thể bị block khi chưa có log mới.

Đây là scenario A của đề: quan sát mutex-protected shared buffer và all-thread backtraces. Kịch bản B inject temperature=45°C không được thực hiện trong log này.

DEBUGGING & PROFILING

9.4. Task 3 — strace -f -c



Hình 4. Top 5 syscalls theo số lần gọi trong khoảng 32 giây (Host mock).

SYSCALL	CALLS	% SYSCALL TIME	LIÊN HỆ VỚI APP
futex	1.015	17,56%	Mutex / condition variable synchronization
mq_timedreceive	958	35,28%	Chờ ALERT/LOG, timed receive
mq_timedsend	958	2,50%	Đưa message vào queues
lseek	639	1,53%	Mock sensor backend /dev/null
clock_nanosleep	637	38,90%	Chờ chu kỳ sampling / main sleep

Tổng cộng 5.286 syscalls và 0,698691 giây trong thống kê syscall-time; con số % biểu thị phần thời gian của các syscall được strace đo, không phải % CPU toàn tiến trình. Việc xuất hiện lseek nhiều lần là đặc điểm của mock sensor trên Host, không nên áp dụng nguyên xi cho ARM character device.

DEBUGGING & PROFILING

9.5. Task 3 — perf sampling và hạn chế

9.5.1. Live application

Lần đầu perf record không chạy được vì kernel perf_event_paranoid=4. Khi thử sudo perf, ứng dụng lại lỗi "open log: Permission denied" và thoát ngay do sự khác biệt quyền sở hữu log. Cuối cùng perf được chạy privileged, trong khi binary sms_app_x86_mock_fixed được chạy dưới user bình thường; điều này tạo được báo cáo live.

PHÂN TÍCH PERF	KẾT QUẢ
Recording	cpu-clock:u, 199 Hz, 34,30 s
Samples	1
Lost samples	0
Symbol xuất hiện	logger_thread (1/1 sample)
Diễn giải	Không đủ mẫu để xếp hạng CPU hotspots đáng tin cậy

9.5.2. perf stat xác nhận workload rất nhẹ

COUNTER	VALUE
task-clock	127,18 ms / 34,14 s
CPUs utilized	0,004
context-switches	2.106
page-faults	765
user CPU time	0,009286 s
sys CPU time	0,024144 s

Tổng quan cho thấy workload phần lớn chờ/sleep, đồng nhất với strace. 100% logger_thread trong báo cáo chỉ phản ánh một sample, KHÔNG chứng minh Logger là điểm nóng thực tế.

9.5.3. Benchmark bổ sung (không phải sms_app live)

Một benchmark CPU-intensive riêng dùng circular_buffer.c và rolling-average đại diện thu 107 samples: process_sms_sample 31,78%, compute_rolling_average 23,36%, cb_push 18,69%, cb_pop 8,41%. Đây là chỉ số của harness, dùng tham khảo khả năng tối ưu, không được ghi như hotspot của ứng dụng chính.

REAL-TIME ANALYSIS

10. Task 4 — RT scheduling implementation

10.1. Chu kỳ tuyệt đối

```
clock_gettime(CLOCK_MONOTONIC, &deadline);
for (;;) {
    deadline += interval; // 100 ms trong benchmark
    clock_nanosleep(CLOCK_MONOTONIC, TIMER_ABSTIME, &deadline, NULL);
    clock_gettime(CLOCK_MONOTONIC, &actual);
    jitter_us = actual - deadline;
}
```

Dùng mốc deadline tăng cố định giảm tích lũy drift của sleep tương đối. Chênh lệch giữa thời điểm actual và expected được chuyển sang microseconds. Mỗi mode lưu tối đa 4096 mẫu jitter; báo cáo trong dữ liệu thực có >1200 mẫu.

10.2. Scheduler và memory locking

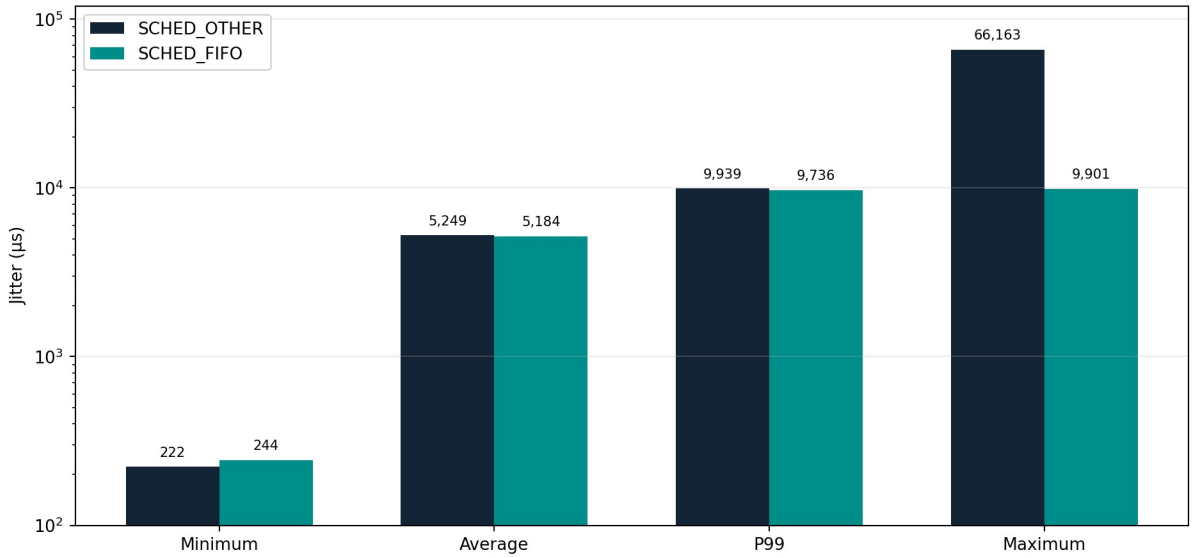
Main dùng pthread_setschedparam() để yêu cầu SCHED_FIFO priority=50 hoặc SCHED_OTHER priority=0 đối với Sensor Thread. pthread_getschedparam() lấy chính sách thực tế; kết quả in requested/effective để tránh báo PASS giả. mlockall(MCL_CURRENT|MCL_FUTURE) được yêu cầu với RLIMIT_MEMLOCK đủ lớn. Nếu thất bại, chương trình ghi mlock=FAILED thay vì coi đã khóa bộ nhớ.

10.3. Phương pháp benchmark

Hai phiên QEMU ARM được chạy với interval=100 ms nhằm nhanh chóng có ≥1000 mẫu cho mỗi chính sách; OTHER n=1249 và FIFO n=1248. Điều kiện QEMU virtualized, không phải board ARM vật lý có PREEMPT_RT, nên phép đo này chỉ phản ánh trường hợp quan sát tại thời điểm thử nghiệm.

REAL-TIME ANALYSIS

10.4. Task 4 — Kết quả jitter



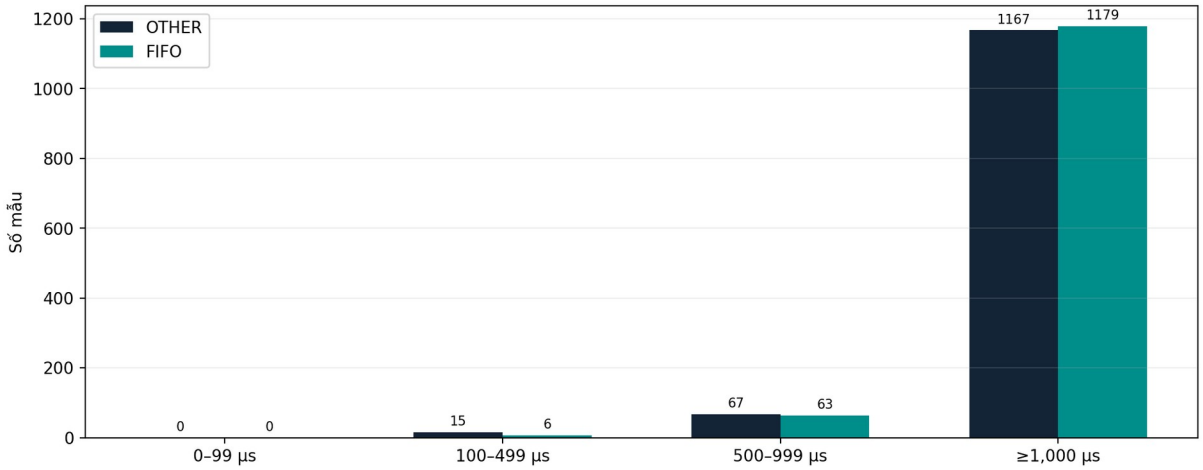
Hình 5. Min/Avg/P99/Max jitter; trục log để biểu diễn chênh lệch lớn giữa các thước đo.

METRIC	SCHED_OTHER	SCHED_FIFO
Samples	1249	1248
Policy / priority	OTHER / 0	FIFO / 50
Minimum (µs)	222	244
Average (µs)	5248,87	5184,17
Maximum (µs)	66.163	9.901
P99 (µs)	9.939	9.736
mlockall	OK	OK

Cả hai phiên report requested và effective trùng nhau, giúp xác nhận chính sách scheduling thật sự được áp dụng. P99 giảm (9939 − 9736)/9939 × 100 = 2,04%; trong khi giá trị max quan sát giảm mạnh nhưng đó là thống kê của mẫu và có thể bị ảnh hưởng bởi một outlier.

REAL-TIME ANALYSIS

10.5. Phân phối jitter và phân tích



Hình 6. Bốn khoảng histogram bắt buộc, 100 ms interval.

JITTER BUCKET	OTHER	FIFO
0–99 µs	0	0
100–499 µs	15	6
500–999 µs	67	63
≥1000 µs	1167	1179

10.6. Đánh giá mức đáp ứng

Khoảng 93–94% samples có jitter ≥1 ms ở cả hai chế độ; điều này cho thấy môi trường QEMU đang có scheduling delay ở mức mili-giây. Nếu yêu cầu cảm biến 500 ms cho phép sai số dưới 1% chu kỳ (=5 ms), kết quả trung bình khoảng 5,2 ms và P99 khoảng 9,7–9,9 ms có thể vượt ngưỡng đề xuất. Ngưỡng 5 ms là ví dụ lựa chọn kỹ thuật trong báo cáo, không phải ngưỡng giảng viên định sẵn hay đặc tả công nghiệp đã xác thực.

10.7. Hướng giảm jitter

- PREEMPT_RT để giảm các vùng không preemptible, sau đó lặp lại phép đo dưới cùng tải.
- CPU isolation / task affinity, pin Sensor Thread, giảm cạnh tranh workload khác.
- IRQ affinity và dùng clocksource ổn định; khóa page faults và hạn chế printf ở đường real-time.
- Thu thập thêm trên phần cứng thật với interval=500 ms và so sánh P99 theo nhiều phiên.

ĐÁNH GIÁ TRUNG THỰC

11. Các điểm đã khắc phục và giới hạn còn lại

11.1. Hồi quy thông lượng 500 ms — đã khắc phục trong lần retest

Bản hardening trước sửa từng tạo 78 mẫu/59,36 giây (~1,31 mẫu/giây), kém so với 2 mẫu/giây lý tưởng ở 500 ms. Sau bản vá retry EAGAIN trên Sensor Thread và build lại ARM binary/initramfs, lần kiểm thử 60 giây mới ghi 116 mẫu/59,54 giây (~1,95 mẫu/giây), exit=0, consumed=116, overflow=0, delta read_count=116. Đây là bằng chứng đủ cho tiêu chí 120±10 mẫu của đề trong phiên mới nhất. Cơ chế EAGAIN là giả thuyết kỹ thuật phù hợp và thay đổi có hiệu quả; chưa có thông số lỗi đọc trước/sau để quy nguyên nhân duy nhất.

11.2. perf hotspot ranking

Một sample cpu-clock:u chứng minh perf có hoạt động, nhưng không đủ để kết luận hàm tiêu tốn CPU lớn nhất. Có thể đo thêm -e cycles:u nếu hardware PMU hỗ trợ hoặc chạy workload có tăng tần suất sampling theo phép thử nhưng vẫn dùng ứng dụng thật; cần tách bạch mọi benchmark nhân tạo.

11.3. Bộ nộp bài đang thiếu những gì

- Video demo 3–5 phút: chưa có trong archive cung cấp.
- initramfs_asgn2.cpio.gz chưa có trong archive report materials này; vẫn nằm ở Ubuntu workspace/output.
- Screenshot diễn giải và log raw perf_sms_live_retry.data chưa có trong tar do permission; text perf report đã được cung cấp.
- Bản báo cáo hoàn chỉnh này phải được đưa vào thư mục bao_cao/ của bộ nộp chính thức; archive tư liệu đã cung cấp trước đó chưa chứa PDF FINAL, initramfs hoặc video demo.

Nội dung báo cáo đã cập nhật bằng phép retest đạt 116/116 mẫu và được chuyển sang phiên bản FINAL. Tuy vậy, trạng thái hoàn thành báo cáo KHÔNG đồng nghĩa hồ sơ nộp LMS đã đầy đủ: video demo 3–5 phút và gói lưu trữ theo cấu trúc yêu cầu cần chuẩn bị riêng. Giới hạn live perf chỉ 1 sample và benchmark jitter ở chu kỳ 100 ms vẫn được công bố rõ.

TASK 5

12. Hướng dẫn demo video 3–5 phút

12.1. Kịch bản đề xuất

THỜI GIAN	CẢNH QUAY	LỜI GIẢI THÍCH
00:00–00:30	Giới thiệu repo, mã SE203134	Vai trò kernel/userspace
00:30–01:15	Boot QEMU, insmod sms_sensor.ko	/dev/sms_sensor, /proc/sms_stats
01:15–02:30	sms_app -i 500 -d 0 -v	4 threads, rolling avg, alerts
02:30–03:10	tail -f /tmp/sms.log	Timestamp và logger rotation
03:10–04:00	Ctrl+C hoặc kill -INT	Graceful shutdown; SUMMARY, exit=0
04:00–05:00	Kết quả Valgrind, jitter, perf	Chỉ rõ limits: RT 100ms, perf sparse

12.2. Các lệnh cho demo QEMU

```
insmod /lib/modules/5.15.0/sms_sensor.ko
ls -l /dev/sms_sensor
cat /proc/sms_stats
/usr/bin/sms_app -i 500 -t 40 -h 80 -d 0 -v
# Ctrl+C sau khi quan sát các thread và alert
cat /proc/sms_stats
tail -15 /tmp/sms.log
```

Phần video cần quay màn hình thực tế kèm lời giải thích của sinh viên. Tài liệu chỉ cung cấp kịch bản, không thay thế video nộp.

TASK 5

13. Reflection — nhìn lại quá trình

13.1. Kỹ năng khó nhất

Trong quá trình thử nghiệm, phần khó nhất là ghép nối môi trường Host Ubuntu với ARM/QEMU và tìm lỗi không xuất hiện ngay ở bước compile. Ví dụ, chương trình x86 không dùng được ARM device thật; thiếu CONFIG_POSIX_MQUEUE từng làm mq_open báo Function not implemented. Sự khác nhau giữa quyền người dùng và root cũng khiến perf recording thành công nhưng app không mở được log. Các lỗi như vậy đòi hỏi kiểm tra có phương pháp ở từng tầng: kernel config → rootfs → library → device interface → thread.

13.2. Nếu làm lại từ đầu

Nên chia chương trình thành module từ đầu, thiết kế API cho sensor backend có mock thay vì vá về sau, và tạo regression tests sẵn cho normal/SIGINT/logger failure/rotation. Các bản build cần có ID hoặc Git commit trong log để tránh nhầm benchmark cũ và source final mới. Việc áp dụng absolute scheduling nên đi kèm counters cho attempts/read_success/EAGAIN để phân biệt jitter và throughput.

13.3. Kỹ năng có giá trị trong thực tế

Quan trọng nhất là đọc log đúng ngữ cảnh, phân biệt user-space với kernel-space, biết dùng GDB backtrace cùng Valgrind/strace/perf, và thiết kế shutdown không bỏ rơi các resource. RT scheduling không được kết luận chỉ từ requested policy mà phải xác nhận effective scheduling.

13.4. Cải tiến nếu còn thời gian

Ưu tiên thêm watchdog, health metrics export và persistent journal; sau đó có thể gửi dữ liệu qua MQTT đến dashboard. Bản vá retry EAGAIN đã giúp kiểm thử 500 ms đạt 116 mẫu trong một lần chạy; khi phát triển sản phẩm nên bổ sung counter EAGAIN/retry, chạy test dài ngày và thực hiện hardware-in-the-loop với cảm biến thật.

TỔNG KẾT

14. Kết luận và tài liệu tham khảo

Assignment 02 minh họa một hệ thống Embedded Linux đa luồng có khả năng đọc cảm biến qua character driver, xử lý trung bình trượt và các loại cảnh báo, ghi log độc lập với timestamp/rotation, quản lý bộ nhớ và thực hiện graceful shutdown. Các kết quả QEMU và Host xác nhận phần lớn chức năng thực thi được: Valgrind sạch, driver ioctl/read đúng, thread synchronization ổn trong các kịch bản thử, hai chính sách RT được áp dụng và có histogram jitter hơn 1.000 mẫu.

Giới hạn còn lại cần công khai: lần benchmark perf live chỉ lấy 1 CPU sample, chưa đủ xếp hạng CPU hotspot; Task 4 dùng chu kỳ 100 ms nhằm thu ≥1.000 mẫu jitter, không phải phép xác minh ở chu kỳ mặc định 500 ms. Hồi quy trước đó 78 mẫu/59,36 giây đã được retest thành công trên bản source cập nhật: 116 mẫu/59,54 giây, exit=0 và không overflow. Cần tiếp tục thử thêm nhiều phiên và trên phần cứng thật để đánh giá độ ổn định ngoài QEMU.

Nguồn dữ liệu và tài liệu

- Assignment 02 – Embedded Linux Systems, handout do người dùng cung cấp (15 trang).
- Source archive SE203134_Asgn02_Report_Materials.tar.gz, 27/10/2026: app/*.c, driver/*.c, logs/*.txt.
- POSIX Programmer’s Manual / Linux man-pages: pthread_mutex_lock(3), pthread_cond_wait(3), mq_receive(3), mq_timedsend(3), clock_nanosleep(2).
- Linux Kernel Documentation: device drivers, procfs / seq_file, PREEMPT_RT design.
- Valgrind User Manual: Memcheck; perf-record(1), perf-report(1), perf-stat(1), strace(1).

PHỤ LỤC

Phụ lục A — Các lệnh tái lập chính

Build driver và app

```
cd ~/embedded_asgn2/driver && make
cd ~/embedded_asgn2/app && make clean && make host && make arm
file driver/sms_sensor.ko app/sms_app app/sms_app_x86
```

Driver / QEMU smoke test

```
insmod /lib/modules/5.15.0/sms_sensor.ko
cat /proc/sms_stats
/usr/bin/sms_app -i 500 -t 40 -h 80 -d 60 -v
echo "exit=$?"; cat /proc/sms_stats
```

Host profiling

```
valgrind --leak-check=full --show-leak-kinds=all ./sms_app_x86_mock -d 0
strace -f -c ./sms_app_x86_mock -d 32
sudo perf record -e cpu-clock:u -F 199 -g -o perf.data -- \
  sudo -u minh ./sms_app_x86_mock_fixed -i 100 -d 35
sudo perf report --stdio -i perf.data
```

QEMU RT experiment

```
sh /usr/bin/task4_run.sh
# Collect [RT_REPORT] and [JITTER_HIST] for OTHER and FIFO
# Ensure effective policy, mlock=OK, n>=1000.
```

PHỤ LỤC

Phụ lục B — Danh mục minh chứng

TASK	LOG / TÊN FILE
Task 1	task1_qemu_session.txt; final_fixes_qemu_session.txt
Task 2	valgrind_sms.txt; memory_T05/T30/T55.txt; task2_driver_integration.txt
Task 2	proc_sms_stats_before/after.txt; process_maps.txt
Task 3	gdb_sms.txt; strace_sms.txt; task3_summary.txt
Task 3	perf_sms_live_retry.txt; perf_sms_stat_final.txt
Task 3	perf_sms_hotpath_benchmark.txt (supplement, not live app)
Task 4	jitter_report.txt; task4_qemu_session.txt
Reliability	final_technical_fix_results.txt; final_host_reliability_summary.txt
Task 1 — final retest	Ảnh Terminal QEMU 27/10/2026: 116/116 mẫu, exit=0, interval=500 ms; Phụ lục C

Kiểm tra tính toàn vẹn archive cung cấp

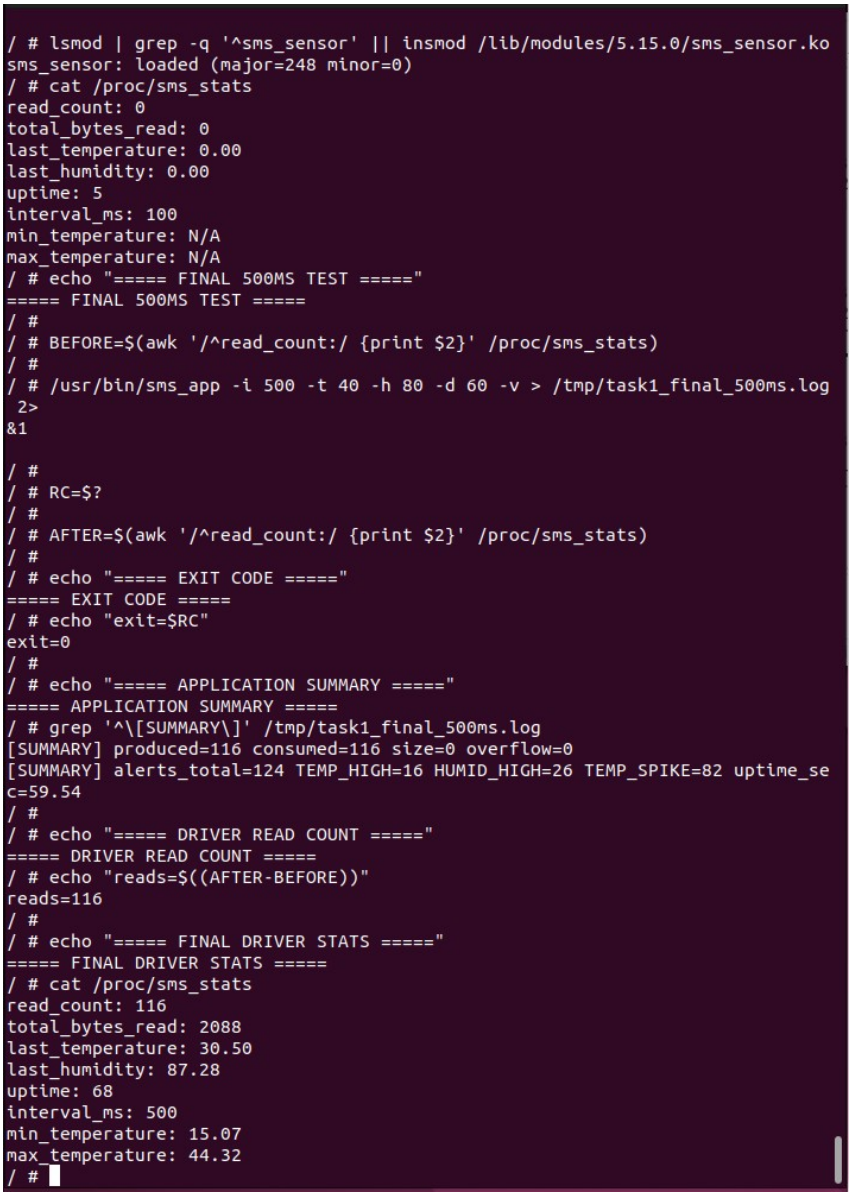
Archive tư liệu SE203134_Asgn02_Report_Materials.tar.gz đã được dùng để đánh giá source/logs; phiên tar ban đầu bỏ sót perf_sms_live_retry.data do permission denied, nhưng báo cáo text perf_sms_live_retry.txt có sẵn. Bằng chứng retest 500 ms ngày 27/10/2026 bổ sung từ screenshot Terminal, xác nhận 116/116 mẫu, exit=0, delta read_count=116. Archive tư liệu KHÔNG phải bộ nộp chính thức vì chưa chứa output/initramfs, video demo và báo cáo PDF FINAL.

Các hành động bắt buộc trước khi nộp

11. Hoàn tất: chạy lại bản đã sửa Sensor Thread tại -i 500 -d 60 trên QEMU; ghi nhận 116/116 mẫu, buffer 0, overflow 0, exit 0, read_count tăng 116 (xem Phụ lục C).
12. Thêm demo mp4 và bản initramfs_asgn2.cpio.gz; xác nhận boot thành công.
13. Xác nhận tên sinh viên, định dạng MSSV_HoTen_Asgn02 và output PDF. Nên lưu checksum và Git commit.
14. Nếu muốn kèm perf raw, sửa quyền file riêng sau khi xác minh owner: dùng sudo cp hoặc chown đúng file.

PHỤ LỤC C — MINH CHỨNG KIỂM THỬ CUỐI

Ảnh chụp console ARM/QEMU ngày 27/10/2026: lệnh -i 500 -d 60; kết quả produced=116, consumed=116, size=0, overflow=0; exit=0; số lần đọc driver tăng 116. Ảnh do sinh viên cung cấp sau bản sửa retry EAGAIN.



Hình C1. Bằng chứng đầu ra kiểm thử cuối — QEMU ARM, chu kỳ 500 ms.